CODE A NOVA

Internship Project Brief • Project 4 • 10-Day Sprint

PYTHON DEVELOPMENT

DataSentinel — Automated Data Quality & Pipeline Monitoring Platform

Build a Python platform that connects to data sources, applies configurable quality rules, detects schema drift and anomalies, generates executive-level HTML reports, and alerts stakeholders — the data quality layer every data team needs.

■ Domain Overview

Domain: Python Development spans CLI tools, web APIs, automation pipelines, and data processing systems. Python engineers at companies like Airbnb, Spotify, and Netflix build the data engineering tooling and backend services that power entire product analytics ecosystems. Enterprise Python engineering demands Pythonic design patterns, type-annotated codebases, comprehensive testing, proper packaging, and modular architecture that can be extended without modification.

Objective: Design and build DataSentinel — a Python CLI platform that connects to CSV, Excel, JSON, PostgreSQL, and REST API data sources; applies configurable data quality rules (null checks, uniqueness, range validation, pattern matching, referential integrity); detects schema drift between pipeline runs; identifies statistical anomalies using Z-score and IQR methods; generates executive-quality HTML and PDF reports; and sends alert summaries via email or Slack webhook — all configurable via YAML rule files with no code changes required.

Expected Outcome: A production-grade, pip-installable Python package with a CLI interface, configurable YAML rule system, multi-source data connectors, statistical anomaly detection, schema drift tracking, and a professional report generator — documented, tested above 75% coverage, and packaged with pyproject.toml.

■■ Tech Stack

Category	Technology
Language	Python 3.12+ with full type annotations (mypy strict)
CLI	Typer with Rich for styled terminal output
Data Processing	Pandas 2.1+ and NumPy
Database Connector	SQLAlchemy 2.0 (PostgreSQL, SQLite)
Configuration	PyYAML for rule definitions, Pydantic v2 for validation
HTML Reports	Jinja2 templates with embedded Chart.js via CDN
PDF Reports	WeasyPrint or ReportLab
Alerting	smtplib for email + httpx for Slack webhook

Testing	pytest + pytest-cov; hypothesis for property-based testing
Packaging	pyproject.toml with Hatch or Poetry

■■ Architecture & Data Flow

- CLI Layer: Typer commands — validate, report, compare-schemas, schedule, list-rules; Rich progress bars and tables
- Connector Registry: Abstract BaseConnector class; concrete adapters for CSV, Excel, JSON, SQLite, PostgreSQL, REST API
- Rule Engine: YAML-driven rule loader; RuleExecutor applying null_check, unique_check, range, regex, foreign_key rules per column
- Schema Registry: Stores column names/types/nullability fingerprints; diffs current schema against registered baseline
- Anomaly Detector: Z-score (|z|>3) and IQR (1.5x fence) per numeric column; flags rows and columns as anomalous
- Report Generator: Jinja2 HTML template with embedded Chart.js charts; WeasyPrint PDF rendering
- Alert Service: Sends issue summaries via SMTP email or Slack webhook with pass/fail counts and top issues
- Scheduler: APScheduler integration for recurring validation runs with cron expression config
- Result Store: SQLite-backed run history for trend analysis across pipeline executions

■ 10-Day Milestone Plan

Day	Goals & Tasks
Day 1	pyproject.toml packaging setup, Typer CLI skeleton, Rich-styled terminal output, GitHub repo with CI (pytest on push)
Day 2	Connector layer — Abstract BaseConnector, CSV and Excel adapters with Pandas, unified DataSet interface
Day 3	JSON adapter, SQLAlchemy PostgreSQL and SQLite connectors, REST API connector with pagination support
Day 4	Rule engine — YAML rule loader with Pydantic v2 validation; null_check, unique_check, range, regex rule executors
Day 5	**Schema registry — baseline schema fingerprint storage; schema drift detection with column add/remove/type-change diffs**
Day 6	Statistical anomaly detection — Z-score and IQR per numeric column; anomalous row flagging with confidence scores
Day 7	**HTML report generator — Jinja2 template, Chart.js charts (pass rate bar, anomaly trend line), executive summary table**
Day 8	PDF report export (WeasyPrint), alert service (SMTP + Slack webhook), APScheduler recurring run integration

Day 9	pytest suite — unit tests for all rules and connectors; hypothesis property-based tests for anomaly detector; achieve >75% coverage
Day 10	**Package publish (PyPI test instance), mypy strict pass, README, 3 sample rule files, 5-min demo recording**

■ Deliverables

1. pip-installable Python package (pyproject.toml + wheel built)
2. GitHub repository with typed Python source, CI pipeline, and semantic commits
3. 3 sample YAML rule configuration files (e-commerce orders, HR data, sales pipeline)
4. 3 sample datasets triggering real quality violations and anomalies
5. Generated HTML report sample (professional quality, charts embedded)
6. Generated PDF report sample
7. pytest coverage report (>75% line coverage)
8. mypy strict mode output (zero errors)
9. Architecture diagram showing connector, rule engine, and report pipeline
10. README with CLI reference, YAML rule authoring guide, and connector setup

■ Evaluation Rubric

Criteria	Weight	Success Indicators
Connector Architecture	15%	Abstract adapter pattern, all 5+ sources working, error handling
Rule Engine Correctness	25%	All rule types working, YAML config drives execution, result accuracy
Schema Drift Detection	10%	Baseline storage, diff accuracy, column type change detection
Anomaly Detection	15%	Z-score and IQR correct, row-level flagging, confidence scores
Report Quality	15%	HTML and PDF professional, charts embedded, executive summary clear
Testing & Type Safety	10%	Coverage >75%, hypothesis tests, mypy strict zero errors
Packaging & Documentation	10%	pip install works, README complete, CLI help text accurate
TOTAL	100%	

■ Feature Checklist

- ■ Typer CLI with Rich-styled output and progress bars
- ■ CSV and Excel data connectors
- ■ JSON flat file and nested path connector
- ■ PostgreSQL and SQLite connectors (SQLAlchemy)
- ■ REST API connector with pagination
- ■ Schema baseline registration and drift diff
- ■ Z-score outlier detection per numeric column
- ■ IQR fence anomaly flagging with confidence score
- ■ Jinja2 HTML report with Chart.js charts
- ■ WeasyPrint PDF report export

- ■ YAML-configured null check rule
- ■ YAML-configured uniqueness check rule
- ■ YAML-configured numeric range validation
- ■ YAML-configured regex pattern matching
- ■ Referential integrity cross-column check
- ■ SMTP email alert with summary
- ■ Slack webhook alert integration
- ■ APScheduler cron-based recurring runs
- ■ SQLite-backed run history for trend analysis

■ Learning Outcomes

- Design a plugin-style connector architecture using Python abstract base classes
- Build YAML-driven configuration systems validated with Pydantic v2
- Apply statistical methods (Z-score, IQR) for data anomaly detection in Pandas
- Generate professional HTML and PDF reports using Jinja2 and WeasyPrint
- Write property-based tests using the Hypothesis library for validation logic
- Package a Python project with pyproject.toml and publish to PyPI test instance
- Enforce full type safety across a Python codebase using mypy strict mode
- Implement alerting integrations with SMTP email and Slack webhooks

■ Resources & References

- Python Type Hints: https://docs.python.org/3/library/typing.html
- Typer: https://typer.tiangolo.com/
- Pandas 2.0 Docs: https://pandas.pydata.org/docs/
- Pydantic v2: https://docs.pydantic.dev/latest/
- SQLAlchemy 2.0: https://docs.sqlalchemy.org/en/20/
- Jinja2: https://jinja.palletsprojects.com/
- WeasyPrint: https://weasyprint.org/
- Hypothesis: https://hypothesis.readthedocs.io/
- mypy: https://mypy.readthedocs.io/
- PyPI Packaging Guide: https://packaging.python.org/en/latest/tutorials/packaging-projects/

■■ Constraints & Guidelines:

- All functions and classes must include type annotations — mypy strict mode must pass with zero errors
- All rule parameters must be defined in YAML — no hardcoded thresholds in Python source
- The connector registry must allow adding new sources without modifying existing connector code (Open/Closed Principle)
- Anomaly detection must operate on datasets up to 500,000 rows without loading all data into memory simultaneously
- HTML reports must render correctly in Chrome, Firefox, and Safari without JavaScript frameworks
- Alert payloads must not include raw data rows — only summary counts and top violations
- All connectors must handle connection failure gracefully with descriptive error messages
- Scheduled runs must not overlap — APScheduler must use coalesce=True for missed runs
- The package must be installable in a clean virtual environment with pip install -e . without errors
- pytest must run in under 60 seconds — use fixtures and mocks to avoid live database connections in unit tests
- All CLI commands must include --help text describing parameters and usage examples
- Sensitive credentials (database passwords, API keys) must be read from environment variables only